Coinsult

Request your audit at **coinsult.net**

Advanced Manual

Smart Contract Audit

October 15, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

Six Seven AI

0x53ce66eB7A54d56D9ac84ef9a44e8a91B153cf7c

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	4	4	0	0
Low-Risk	5	5	0	0
Medium-Risk	2	2	0	0
Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way
Low-Risk	Won't cause any problems, but can be adjusted for improvement
Medium-Risk	Will likely cause problems and it is recommended to adjust
Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

Six Seven AI	
Website	sixsevenai.netlify.app
Blockchain	BNB Smart Chain (chain ID 56)
Source Code	contracts/SixSevenStaking.sol
Contract Address	0x53ce66eB7A54d56D9ac84ef9a44e8a91B153cf7c

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview 2
- Manual Code Review 2

Audit Summary 2

Audit Scope 3
- Audit Method 3
- Automated Vulnerability Check 3
- Manual Code Review 3
- Used tools 3

Table of Contents 4

The Deployment Note Describes The Token Tax Incorrectly

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
STK-I1	The header warns that deposits and withdrawals would be hit by the nine percent tax unless the contract is exempted. The token only charges that tax when one side of a transfer is a registered exchange pair, so an ordinary transfer in or out of this contract would never have been taxed. The real exposure from losing the exemption is the wallet limit covered elsewhere in this report, not the tax.

Code Snippet

```
// token _update()
bool buy = isPair[from];
bool sell = isPair[to];
if (buy || sell) { fee = amount * (burnFeeBps + treasuryFeeBps) / 10000; }
// neither side is a pair for a staking deposit or withdrawal
```

Recommendation

Correct the note so operators understand that the exemption matters for the wallet limit rather than for the tax.

The Contract Has No Bronze Threshold

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
STK-I2	The website presents three tiers beginning at twenty thousand tokens, but the contract knows only two thresholds. Anything below eighty thousand earns the base rate, including a single token, so there is no on chain difference between a twenty thousand token position and a one token position.

Code Snippet

```
function tierMult(uint256 amount) public view returns (uint16) {
    if (amount >= tierGold) return 200;
    if (amount >= tierSilver) return 150;
    return 100;
}
```

Recommendation

Either add the lowest threshold to the contract or make clear in the interface that the first tier carries no reward multiplier.

Funding The Reward Pool Is Entirely Discretionary

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
STK-I3	Depositing is the only route for native coin into this contract and there is no schedule, no minimum and no commitment of any kind written into the code. At the reviewed block the lifetime total deposited is 0.01 BNB and the contract holds none, while the website states that the pool is refilled with about one thousand dollars every Monday.

Code Snippet

```
totalRewardsFunded = 10000000000000000     (0.01 BNB)
contract BNB balance = 0
totalStaked = 20000 tokens,  totalWeight = 20000
```

Recommendation

Present the reward rate as discretionary rather than as a fixed weekly amount, or hold the commitment in a contract that releases on a schedule.

A Failed Reward Payment Does Not Block Unstaking

Informational *- Does not compromise the functionality of the contract in any way* **[Status: Pending]**

Error Code	Description
STK-I4	If the native coin transfer to a staker fails, the contract puts the reward back on their balance and carries on instead of reverting, so a staker whose address cannot receive native coin can still take their capital out and claim later. The reward accounting was traced through deposits, withdrawals, claims and tier changes and the amount owed never exceeds the amount deposited.

Code Snippet

```
(bool ok, ) = payable(msg.sender).call{value: r}("");
if (ok) { emit Claimed(msg.sender, r); }
else { u.rewards = r; }
```

Recommendation

No change needed. This is the correct handling and it is worth keeping if the contract is ever revised.

A Single Staker Can Capture An Entire Reward Deposit

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
STK-L1	Each deposit is divided by the total weight at the moment it arrives, and there is no minimum stake. Whoever happens to be the only staker when the owner funds the pool receives the whole amount, however small their position, so one token staked at the right moment can take an entire weekly distribution.

Code Snippet

```
function fund() external payable onlyOwner {
    require(msg.value > 0, "no BNB");
    require(totalWeight > 0, "no stakers");
    accRewardPerWeight += msg.value * ACC / totalWeight;
```

Recommendation

Require a minimum stake, and hold the first deposit back until the total weight has passed a sensible threshold.

A Tier Change Only Reaches Users Who Transact Afterwards

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
STK-L2	Changing the thresholds takes effect immediately for new calculations, but each user's stored weight is only recomputed the next time they stake or withdraw. Claiming does not recompute it, although the comment on the setter says it does. Until a user interacts they keep their old weight while sharing the pool with users who have already been updated, which gives a quiet advantage to whoever moves first after every change.

Code Snippet

```
/// Les poids existants se recalculent au prochain
/// stake/withdraw/claim de chaque utilisateur.

// claim() calls _harvest and resets rewardDebt but never
// calls _updateWeight, so the comment is not accurate
```

Recommendation

Recompute the weight inside claim as well, or state plainly that a tier change applies from each user's next deposit or withdrawal.

Leftover Reward Dust Can Never Be Recovered

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
STK-L3	Each deposit is divided by the total weight and the remainder is dropped, so a little of every funding round stays behind. The contract has no receive function, no way to withdraw native coin and no sweep, so that residue builds up and is locked in place permanently. The amounts are small, and they only ever grow.

Code Snippet

```
accRewardPerWeight += msg.value * ACC / totalWeight;  // remainder lost
// no receive(), no fallback, and rescueERC20 covers tokens only
```

Recommendation

Carry the remainder forward into the next deposit, or let the owner sweep anything above the sum still owed to stakers.

Staking Tokens Sent In By Mistake Are Stuck Forever

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
STK-L4	The rescue function deliberately refuses the staking token, which is the right instinct because it stops the owner reaching staked balances. It also means that any $67AI sent to the contract directly, rather than through the staking function, can never be retrieved by anyone. The contract tracks deposits by measuring the balance change, so a stray transfer is not credited to anybody.

Code Snippet

```
function rescueERC20(address token_, uint256 amount_) external onlyOwner {
    require(token_ != address(stakingToken), "cannot touch staked token");
    IERC20(token_).safeTransfer(owner(), amount_);
}
```

Recommendation

Allow the owner to withdraw only the balance above totalStaked, which keeps staked funds untouchable while making stray transfers recoverable.

Ownership Sits On One Key, Although Its Powers Are Narrow

Low-Risk *- Won't cause any problems, but can be adjusted for improvement* **[Status: Pending]**

Error Code	Description
STK-L5	The owner can fund the pool, change the tier thresholds and rescue tokens other than the staking token. It cannot reach staked balances or accrued rewards, which is a good result. The owner address is an ordinary account carrying a delegation to a MetaMask smart account implementation rather than a multiple signature wallet, so one private key stands behind every one of those actions.

Code Snippet

```
owner = 0xd3667ea6a418f4c482d4faa523ae9a97fc889f46
code  = 0xef010063c0c19a282a1b52b07dd5a65b58948a07dae32b
        EIP-7702 delegation to EIP7702StatelessDeleGator
```

Recommendation

Move ownership to a multiple signature wallet before the pool holds meaningful value, and publish the signer set.

● The Lock Multiplier Keeps Paying Long After The Lock Has Expired

Medium-Risk *- Will likely cause problems and it is recommended to adjust* **[Status: Pending]**

Error Code	Description
STK-M1	The duration multiplier is written into the user's weight when they stake and is never revisited. Once the lock end passes, nothing resets it, so a position that chose 42 days keeps earning at twice the rate indefinitely while being free to leave at any moment. A staker who locked once a year ago therefore outranks someone who has just committed to a fresh 42 day lock, and the stated rule that locking is required in order to earn no longer holds for them.

Code Snippet

```
// _updateWeight is only reached from stake() and withdraw()
uint256 w = u.amount * uint256(u.durationMult)
    * uint256(tierMult(u.amount)) / 10000;

// nothing anywhere compares block.timestamp against u.lockEnd
// when the weight is computed
```

Recommendation

Treat an expired lock as the base rate by recomputing the multiplier from the remaining lock time, so a staker has to renew the commitment to keep the bonus.

● Withdrawals Depend On A Token Setting Held By A Different Key

Medium-Risk *- Will likely cause problems and it is recommended to adjust* **[Status: Pending]**

Error Code	Description
STK-M2	The contract works today because the token marks it exempt from both the transfer checks and the wallet limit, and those two flags are set by the token owner, which is a different address from this contract's owner. If the fee exemption is removed, every withdrawal starts running the token's maximum wallet check against the person receiving the tokens, so a staker whose own wallet already holds the 400,000 token limit would be unable to take their principal back out until they moved tokens elsewhere.

Code Snippet

```
// token _update(), reached once the staking contract is no longer exempt
if (!isLimitExempt[to]) {
    require(balanceOf(to) + amount <= maxWallet, "max wallet exceeded");
}

// reviewed state
isFeeExempt[staking]   = true      token owner  0x59603d01...
isLimitExempt[staking] = true      staking owner 0xd3667ea6...
```

Recommendation

Have the token owner confirm in writing that both exemptions are permanent, or mark the staking contract as a protected address in the token so the flags cannot be cleared for it.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.